# Parallel Gaussian Elimination OpenMPI Implementation

CPEN 512 – UBC

ADIN MAUER

NOVEMBER 27, 2024

# Overview

Review of OpenMPI

Strategy for Gaussian Elimination

Results

Discussion +Future Improvements

# Review

MPI – Message Passing – Manages N processes in parallel that communicate via message passing.

Begin MPI by getting RANK and SIZE

```
MPI_Init(&argc, &argv);


int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Size = Number of processes

Rank = Number/ID of this process (in this communicator..)

# Review

Control the flow of the program for different processes depends on the rank. Example:

```
if(rank == 0) {
    communicator_send_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 Sent rows\n");
    #endif
    communicator_recv_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 received rows\n");
    #endif
} else {
```

I will go into more MPI methods later. First.. The algorithmic approach.

# Gaussian Elimination Strategy

Dividing all of the rows into M chunks and distribute them to M processes.

There are N pivots. Define a stage for every pivot. For every stage:

- If this process is the process that has the pivot, it ***broadcasts*** the pivot row to all other processes that have chunks below the *current chunk*. This process them eliminates its own rows below the pivot.

- If this process is the process with a chunk below the pivot, then the chunk eliminate receives the broadcast of the pivot row and then eliminates its own rows.

All chunks operate in parallel.

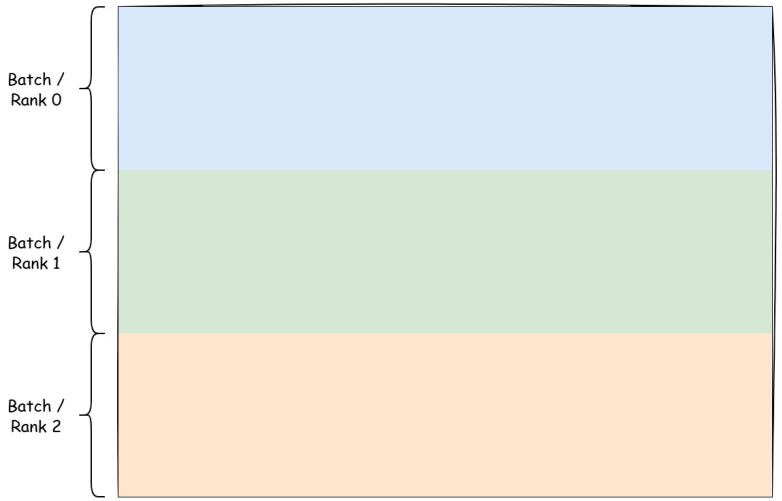
# Data distribution

**Communicator Process (rank 0)**

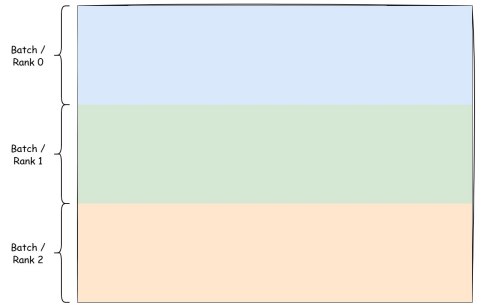
```
if(rank == 0) {
    communicator_send_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 Sent rows\n");
    #endif
    communicator_recv_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 received rows\n");
    #endif
} else {
```

**Eliminator Processes (rank > 0)**

```
DATA_TYPE* rows_to_eliminate =
    intialize_mem4rows2eliminate(rows_to_eliminate, n, rows_this_process);

eliminator_recv_rows(rows_to_eliminate, n, rows_this_process, rank);

eliminate_rows(rows_to_eliminate, n, batch, rows_per_process, rows_this_process, num_batches, is_las

eliminator_send_rows(rows_to_eliminate, n, rows_this_process, rank);
free(rows_to_eliminate);
```



# Data distribution




## Communicator Process (rank 0)

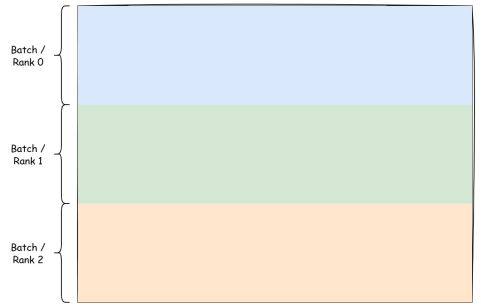
```
if(rank == 0) {
    communicator_send_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 Sent rows\n");
    #endif
    communicator_recv_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 received rows\n");
    #endif
} else {
```

## Eliminator Processes (rank > 0)

```
DATA_TYPE* rows_to_eliminate =
    intialize_mem4rows2eliminate(rows_to_eliminate, n, rows_this_process);

eliminator_recv_rows(rows_to_eliminate, n, rows_this_process, rank);

eliminate_rows(rows_to_eliminate, n, batch, rows_per_process, rows_this_process, num_batches, is_las

eliminator_send_rows(rows_to_eliminate, n, rows_this_process, rank);
free(rows_to_eliminate);
```

```
void communicator_send_rows(DATA_TYPE *U, int n, int num_batches, int rows_per_process, int rows_last_process) {
    for(int i = 0 ; i < num_batches ; i ++) {
        int rows_to_send = (i + 1 == num_batches) ? rows_last_process : rows_per_process;
        MPI_Send(U + rows_per_process * i * n, n * rows_to_send, MPI_FLOAT, i + 1, i + 1, MPI_COMM_WORLD);
    }
}

void communicator_recv_rows(DATA_TYPE *U, int n, int num_batches, int rows_per_process, int rows_last_process){
    for(int i = 0 ; i < num_batches ; i ++) {
        int rows_to_receive = (i + 1 == num_batches) ? rows_last_process : rows_per_process;
        MPI_Recv(U + rows_per_process * i * n, n * rows_to_receive, MPI_FLOAT, i + 1, i + 1, MPI_COMM_WORLD, MPI
    }
}
```

# MPI_Send




## MPI_Send

Performs a blocking send

### Synopsis

```
int MPI_Send(const void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)
```

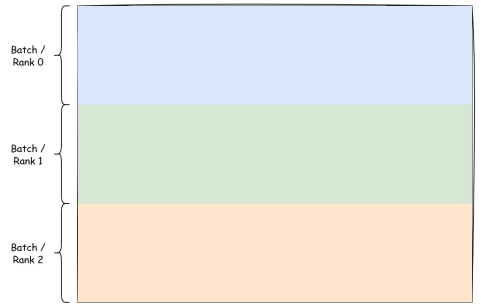
## MPI_Recv

Blocking receive for a message

### Synopsis

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag,
             MPI_Comm comm, MPI_Status * status)
```

# Data distribution




## Communicator Process (rank 0)

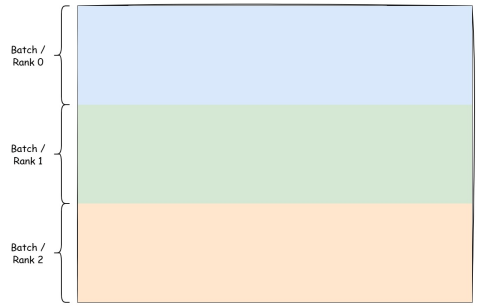
```
if(rank == 0) {
    communicator_send_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 Sent rows\n");
    #endif
    communicator_recv_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 received rows\n");
    #endif
} else {
```

## Eliminator Processes (rank > 0)

```
DATA_TYPE* rows_to_eliminate =
    intialize_mem4rows2eliminate(rows_to_eliminate, n, rows_this_process);

eliminator_recv_rows(rows_to_eliminate, n, rows_this_process, rank);

eliminate_rows(rows_to_eliminate, n, batch, rows_per_process, rows_this_process, num_batches, is_las

eliminator_send_rows(rows_to_eliminate, n, rows_this_process, rank);
free(rows_to_eliminate);
```

```
void eliminator_recv_rows(DATA_TYPE *rows_to_eliminate, int n, int rows_this_process, int rank) {
    int recv_size = n * rows_this_process;
    MPI_Recv(rows_to_eliminate, recv_size, MPI_FLOAT, 0, rank, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    #if DEBUG_MPI_PRINT_DISTS
        print_vector_mpi(rows_to_eliminate, recv_size, rank);
    #endif
}
void eliminator_send_rows(DATA_TYPE* rows_to_eliminate, int n, int rows_this_process, int rank) {
    int send_size = n * rows_this_process;
    #if DEBUG_MPI_PRINT_DISTS
        print_vector_mpi(rows_to_eliminate, send_size, rank);
    #endif
    MPI_Send(rows_to_eliminate, send_size, MPI_FLOAT, 0, rank, MPI_COMM_WORLD);
}
```

# Data distribution




## Communicator Process (rank 0)

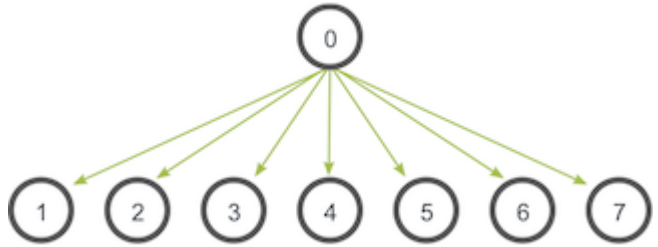
```
if(rank == 0) {
    communicator_send_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 Sent rows\n");
    #endif
    communicator_recv_rows(U, n, num_batches, rows_per_process, rows_last_process);
    #if DEBUG_MPI
    printf("Rank 0 received rows\n");
    #endif
} else {
```

## Eliminator Processes (rank > 0)

```
DATA_TYPE* rows_to_eliminate =
    intialize_mem4rows2eliminate(rows_to_eliminate, n, rows_this_process);

eliminator_recv_rows(rows_to_eliminate, n, rows_this_process, rank);

eliminate_rows(rows_to_eliminate, n, batch, rows_per_process, rows_this_process, num_batches, is_las

eliminator_send_rows(rows_to_eliminate, n, rows_this_process, rank);
free(rows_to_eliminate);
```

```
void eliminator_recv_rows(DATA_TYPE *rows_to_eliminate, int n, int rows_this_process, int rank) {
    int recv_size = n * rows_this_process;
    MPI_Recv(rows_to_eliminate, recv_size, MPI_FLOAT, 0, rank, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    #if DEBUG_MPI_PRINT_DISTS
        print_vector_mpi(rows_to_eliminate, recv_size, rank);
    #endif
}
void eliminator_send_rows(DATA_TYPE* rows_to_eliminate, int n, int rows_this_process, int rank) {
    int send_size = n * rows_this_process;
    #if DEBUG_MPI_PRINT_DISTS
        print_vector_mpi(rows_to_eliminate, send_size, rank);
    #endif
    MPI_Send(rows_to_eliminate, send_size, MPI_FLOAT, 0, rank, MPI_COMM_WORLD);
}
```

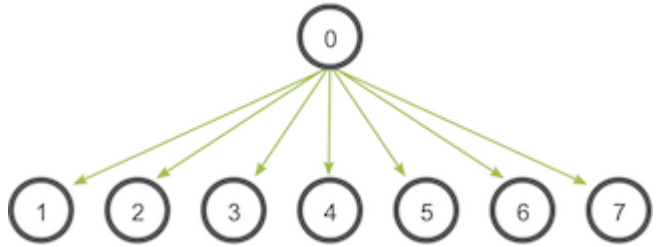
# Row Elimination – Bcast pivot




```
if(is_pivot_batch == 1) {
    count_pivot_rows++;
    memcpy(pivot_row, rows_to_eliminate + pivot_addr_offset_in_batch, n * sizeof(DATA_TYPE));
    pivot_value = pivot_row[pivot_row_index];
    #if DEBUG_MPI2
    printf("A: Batch: %d, pivot_value: %f\n", batch, pivot_value);
    print_vector_mpi(pivot_row, n, batch);
    #endif
}
```

```
MPI_Bcast(pivot_row, n, MPI_FLOAT, pivot_batch_index, elim_comm);
MPI_Bcast(&pivot_value, 1, MPI_FLOAT, pivot_batch_index, elim_comm);
```

# Row Elimination – Bcast pivot




```
MPI_Bcast(pivot_row, n, MPI_FLOAT, pivot_batch_index, elim_comm);
MPI_Bcast(&pivot_value, 1, MPI_FLOAT, pivot_batch_index, elim_comm);
```

**BLOCKING broadcast – root cannot proceed before all processes pick up the broadcast.**

**** Aka, the index of the batch (mpi_process) that HAS the pivot acts as root communicator in the broadcast**

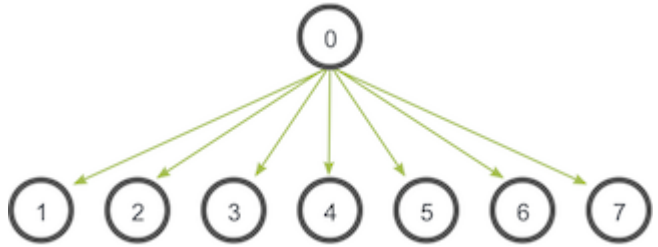
## MPI_Bcast

Broadcasts a message from the process with rank "root" to all other processes of the communicator

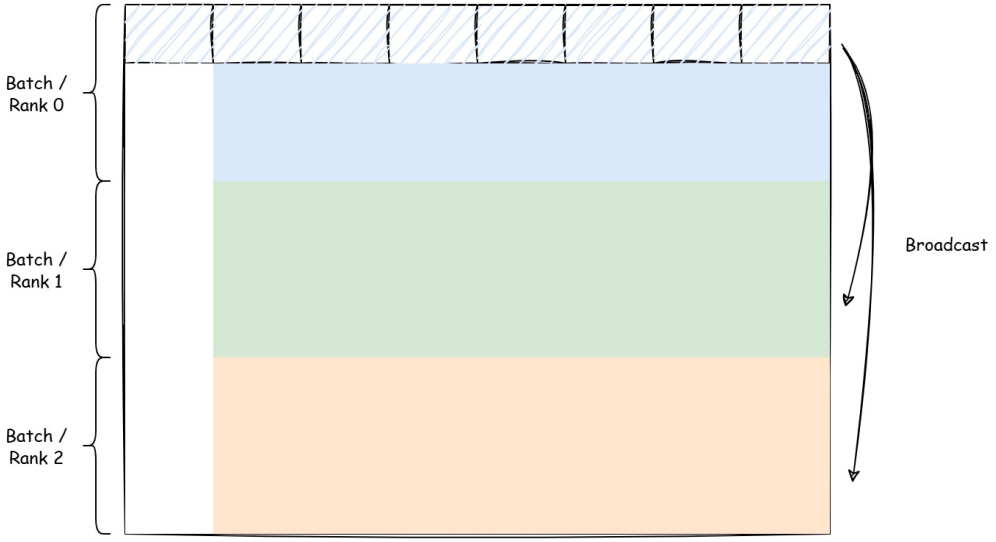
## Synopsis

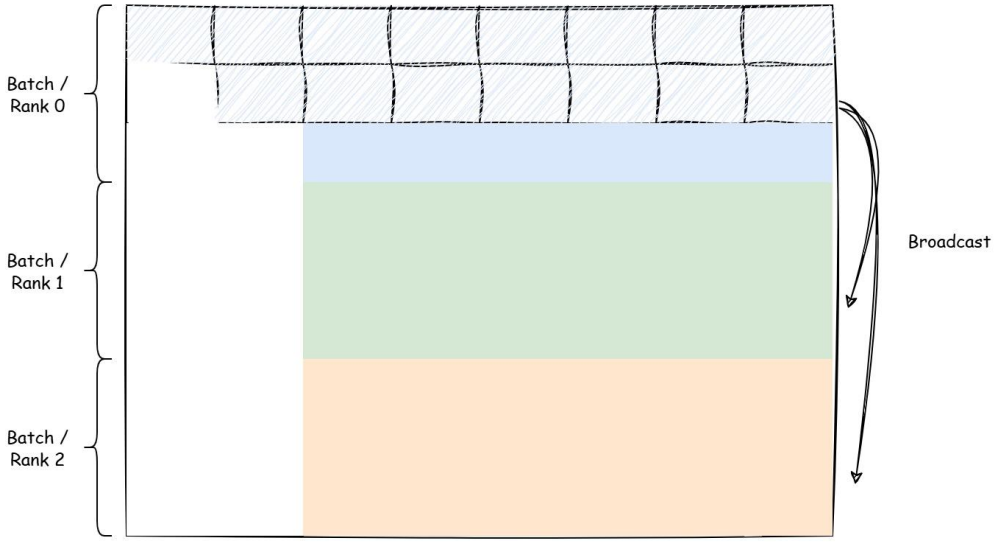
```
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm)
```

# Row Elimination – Bcast pivot

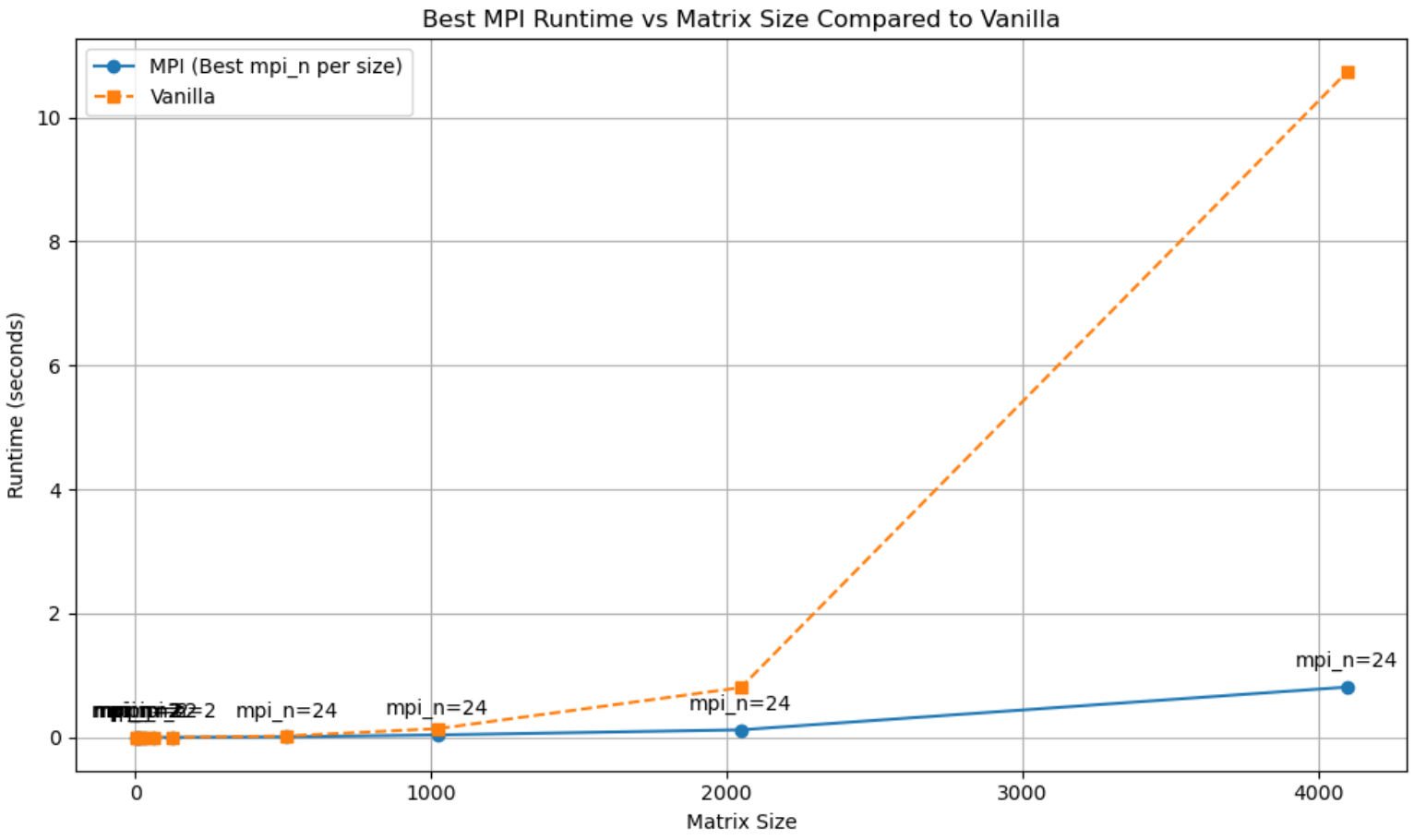



```
MPI_Bcast(pivot_row, n, MPI_FLOAT, pivot_batch_index, elim_comm);
MPI_Bcast(&pivot_value, 1, MPI_FLOAT, pivot_batch_index, elim_comm);
```

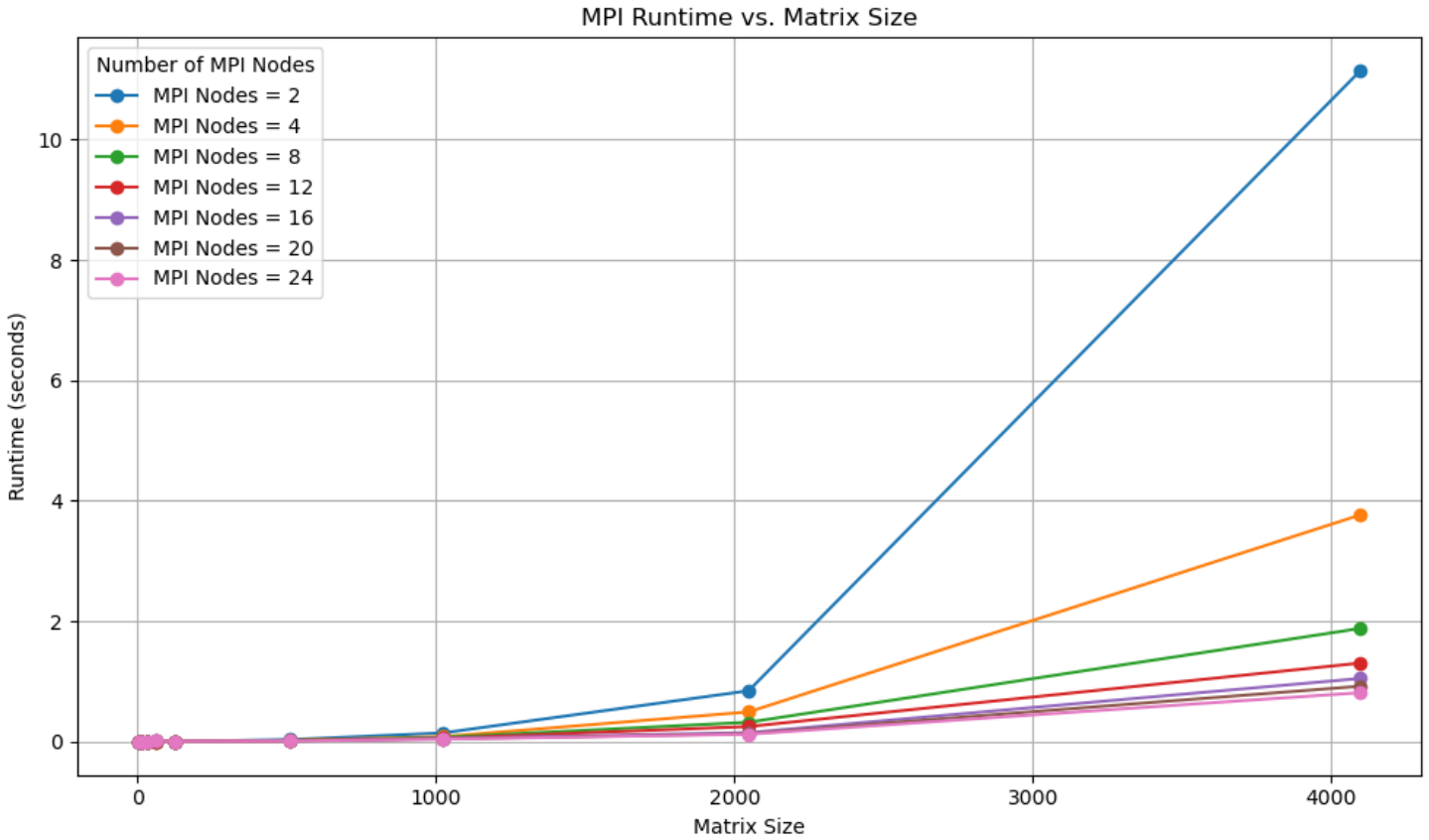


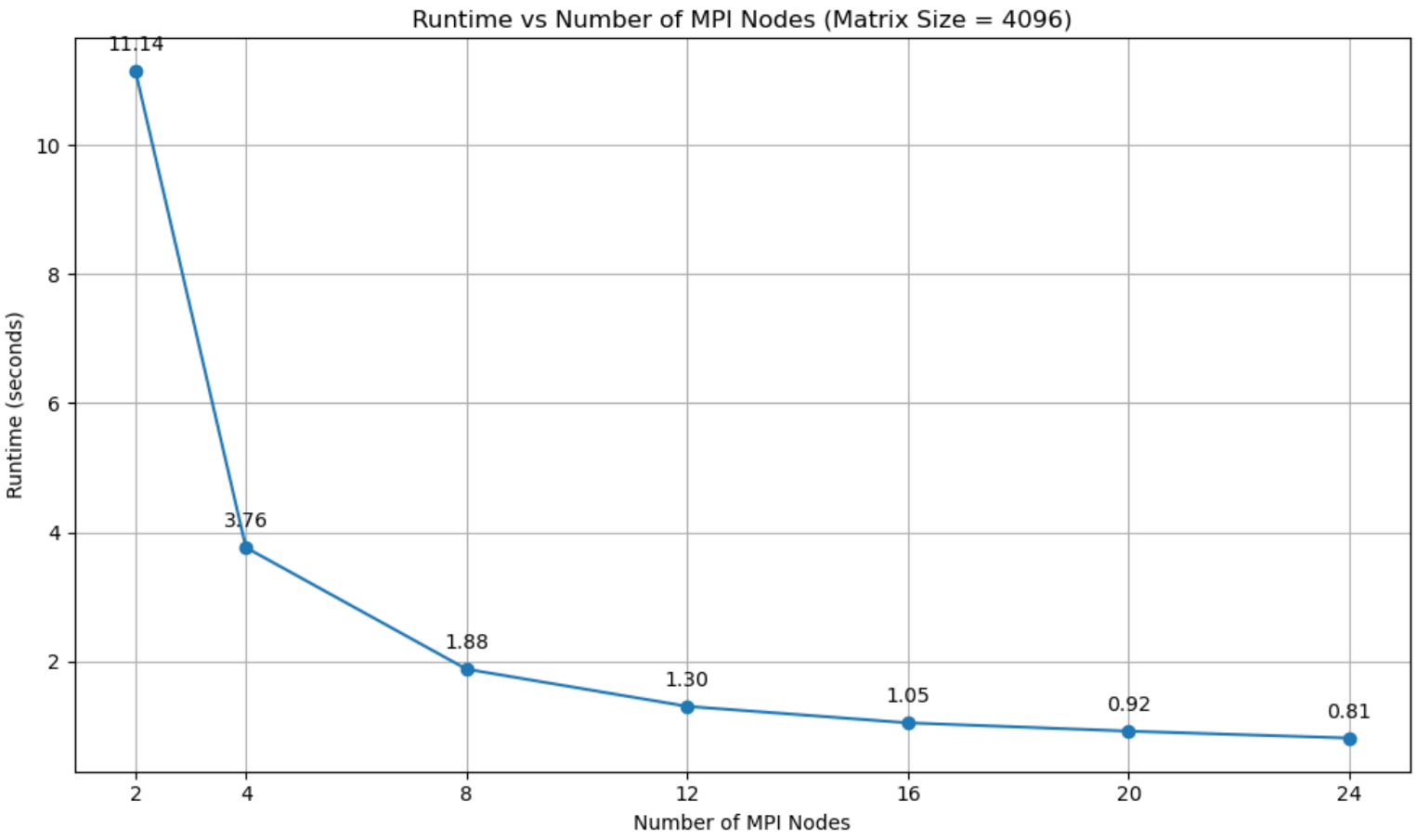
# Results


Best MPI Runtime vs Matrix Size Compared to Vanilla
MPI (Best mpi_n per size)
Vanilla
Runtime (seconds)
Matrix Size
0
2
4
6
8
10
0
1000
2000
3000
4000
mpi_n=24
mpi_n=24
mpi_n=24
mpi_n=24

# Results


MPI Runtime vs. Matrix Size
Number of MPI Nodes
MPI Nodes = 2
MPI Nodes = 4
MPI Nodes = 8
MPI Nodes = 12
MPI Nodes = 16
MPI Nodes = 20
MPI Nodes = 24
Runtime (seconds)
0
2
4
6
8
10
0
1000
2000
3000
4000
Matrix Size

# Results


Runtime vs Number of MPI Nodes (Matrix Size = 4096)
Runtime (seconds)
Number of MPI Nodes
11.14
3.76
1.88
1.30
1.05
0.92
0.81
2
4
6
8
10
2
4
8
12
16
20
24

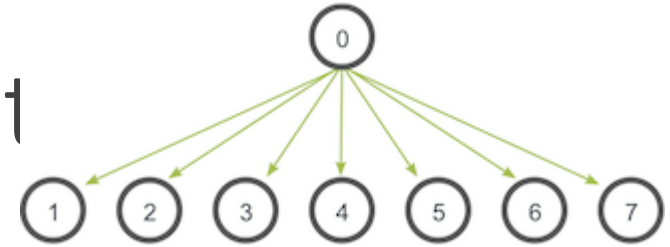
# Results

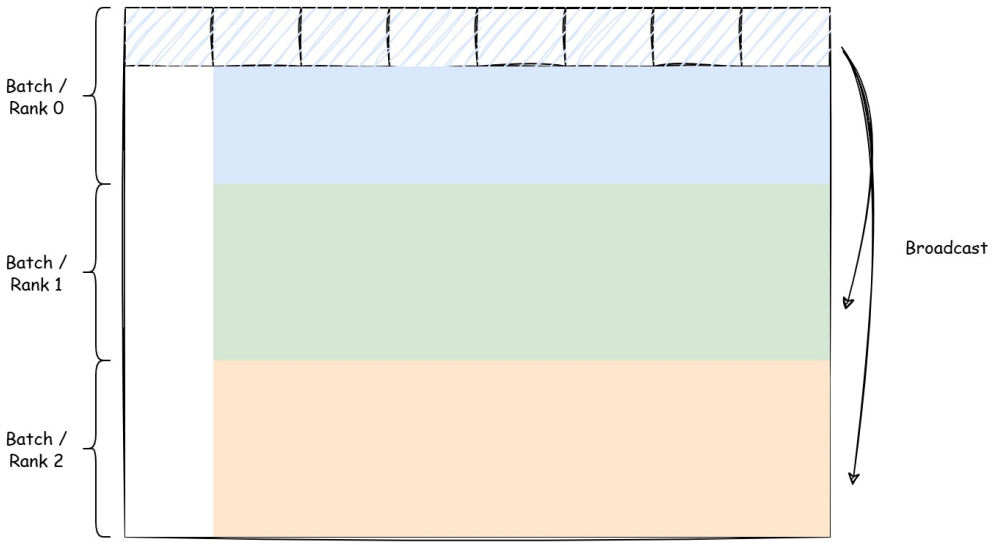
- Best Vanilla runtime for 4096x4096 is 1**0.7 seconds**
- The best runtime for MPI 4096x4096 with **24 processes** is **0.8 seconds**, which is a speedup of **13.24**.

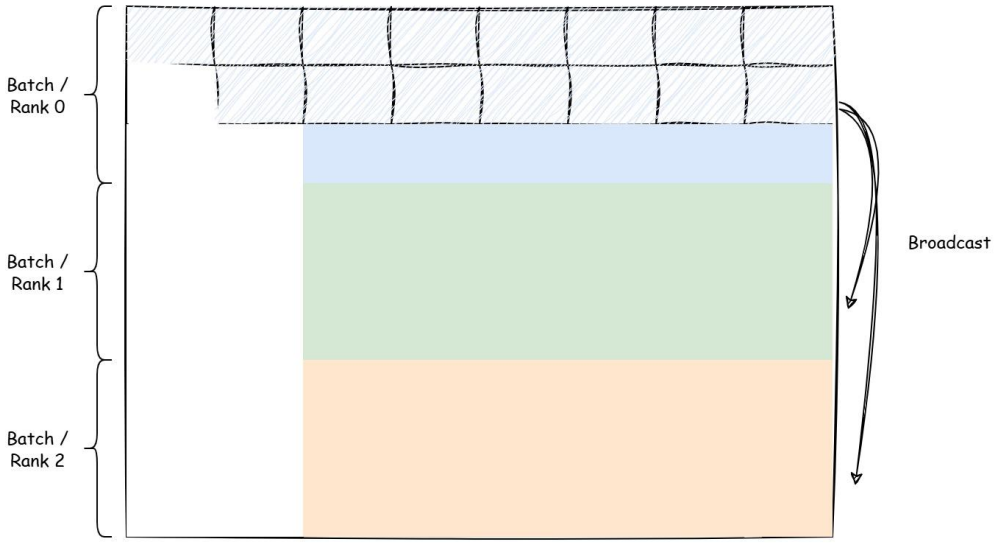
**As of this morning:** speedup of 13.5!

```
Found equivalent - new time: 0.788517 - old seconds: 0.810297
```

# Potential Improvements and thought
# Non-blocking broadcasting



# Potential Improvements and thoughts Non-blocking broadcasting

One idea for improvement is to use non-blocking broadcasting.

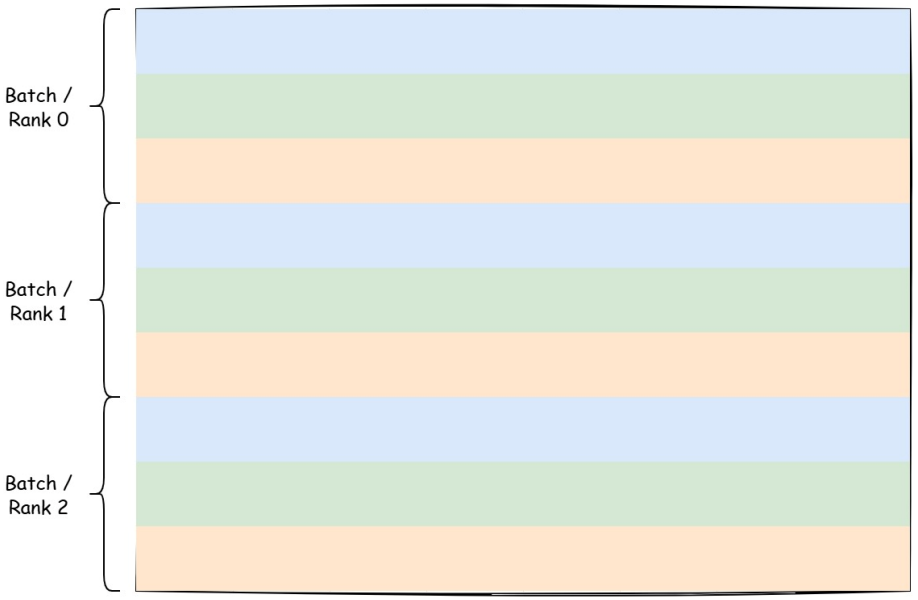
This can be done using MPI_Ibcast, but it is complicated for several reasons:

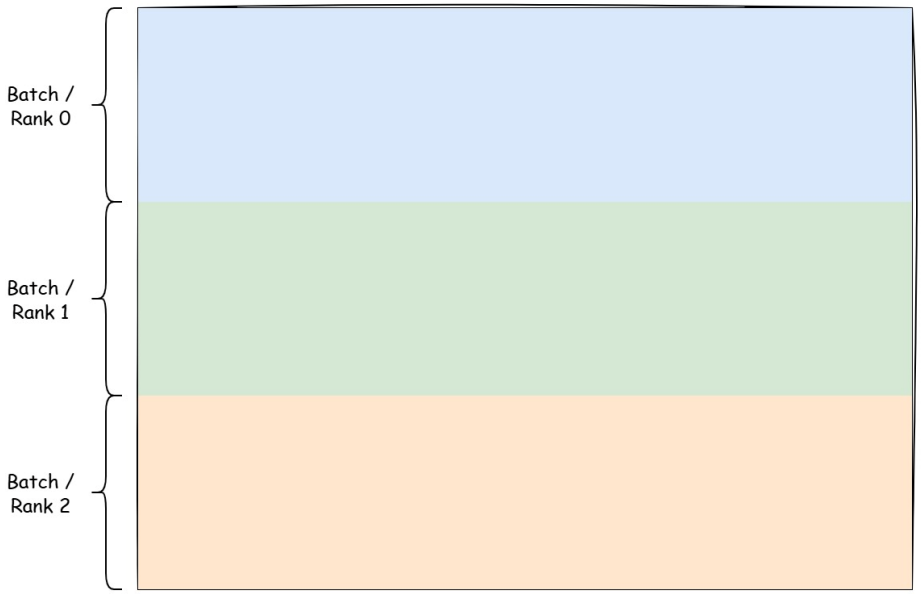
* Both the root broadcasting process and listener processes see MPI_Ibcast as non-blocking, but the desired functionality is that only the communicator will not be blocked, but the listener proceeds only after receiving the data.

* This can be achieved using the MPI_Test function which tests in the given process has already received the buffer from a non-blocking broadcast or not.

* Although the root process can be non-blocking, after eliminating all of its rows it must wait for the listener to pick up the buffer before free'ing the buffer.

# Potential Improvements and thoughts
# Non-contiguous rows distribution




**VS**